Token Optimization & System Architecture Analysis

Comparative Study on LLM Query Efficiency in Project Workspaces

Executive Summary:
This report evaluates the token consumption, context scoping, and execution efficiency of three distinct Large Language Model (LLM) querying strategies. The objective is to identify the most scalable approach for ongoing system maintenance and development. Analysis confirms that utilizing a dedicated, highly condensed architectural context file (Method 2) yields the most performant results, reducing input token overhead by up to 45% while resolving queries in minimal steps.

1. Testing Methodology

The comparative test was conducted on a codebase (PDF-JSON project), utilizing the Qwen 3.8 Max model. Three context-provisioning methods were evaluated to determine viability for active system development environments:

- **Method 1: Full Project Map Directory** (`@.../project-map/`) – Triggers broader graphify tool skills and file traversal routines.
- **Method 2: Compact Context File** (`@.../02-LLM-Context.md`) – Directly targets a pre-compiled, highly condensed markdown summary.
- **Method 3: Full Repository Context** (`@.../`) – Ingests the entire workspace environment blindly.

2. Performance Metrics Comparison

Metric	Method 1 (Full Map)	Method 2 (Context File)	Method 3 (Full Repo)
Execution Steps	9	**4**	4
Input Tokens	32,303	**17,677**	25,012
Output Tokens	1,744	**1,099**	1,093
Reasoning Tokens	1,598	1,879	2,076
Cache Read (Tokens)	180,608	**62,720**	76,800
Cache Hit Rate	84.8%	78.0%	75.4%

3. Architectural Analysis

Method 2: Compact Context File (Optimal Strategy)

By pointing queries directly to the `02-LLM-Context.md` file, the system operates at peak efficiency. It bypasses unnecessary directory parsing and tool-looping, lowering input token footprint to just 17.7K. For continuous system maintenance and scaling large projects, maintaining this "living" document ensures fast, accurate information retrieval without exhausting the context window.

Method 3: Full Repository Scope (Scalability Risk)

While feeding the entire directory (25.0K input tokens) performs adequately in this localized test, it presents long-term risks. As sub-systems—such as independent medical platforms, athlete management systems, and eLearning modules—are integrated into a centralized dashboard architecture, relying on full-repository ingestion will cause rapid context window bloating and severe latency spikes.

Method 1: Full Map Directory (High Tool Overhead)

Referencing the broader map directory resulted in the highest overhead (32.3K input tokens) and doubled the processing steps (9 steps). The LLM spends valuable processing power traversing intermediate graph files and evaluating tool execution paths. This heavy-duty approach should be strictly

reserved for macro-level codebase audits or major synchronization updates, not targeted daily diagnostic queries.

Strategic Recommendation

To optimize agile development and streamline future system integration efforts, embed the **Method 2** architecture into the standard workflow. By utilizing a centralized `02-LLM-Context.md` file—acting as a centralized dashboard for LLM reasoning—teams can maintain rapid diagnostic speeds, dramatically cut token waste, and effectively manage complex integrations across multiple platforms without degrading AI performance.